

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №6

По вычислительной математике

Вариант 466468

Выполнил:

Ларионов Владислав Васильевич

Группа Р3209

Проверил:

Рыбаков Степан Дмитриевич

Санкт-Петербург, 2026

Содержание

Задание	3
Программа.....	4
Вывод:	8

Задание

Цель лабораторной работы: решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.

№ варианта задания лабораторной работы определяется как номер в списке группы согласно ИСУ.

1. Порядок выполнения работы

2. В программе численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должен быть реализован в виде отдельного класса /метода/функции;
3. Пользователь выбирает ОДУ вида $y' = f(x, y)$ (не менее трех уравнений), из тех, которые предлагает программа;
4. Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
5. Для исследования использовать одношаговые методы и многошаговые методы (см. табл.1);
6. Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
7. Для оценки точности одношаговых методов использовать правило Рунге;
8. Для оценки точности многошаговых методов использовать точное решение задачи: $\varepsilon = \max_{0 \leq i \leq n} |y_{i\text{точн}} - y_i|$;
9. Построить графики точного решения и полученного приближенного решения (разными цветами);
10. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных.
11. Проанализировать результаты работы программы.

2. Требования и содержание отчета

Отчет должен содержать следующие разделы:

- Титульный лист,
- Цель работы,
- Описание алгоритма решения задачи,
- Рабочие формулы используемых методов,

- Листинг программы (по крайней мере, коды используемых методов),
- Скриншоты результатов выполнения программы при различных исходных данных (не менее трех),
- Графики точного решения и полученного приближенного решения,
- Выводы.

3. Варианты задания

Одношаговые методы:

1. Метод Эйлера,
2. Усовершенствованный метод Эйлера,
3. Метод Рунге-Кутты 4-го порядка.

Многошаговые методы (методы предиктор-корректор):

4. Адамса,
5. Милна.

Программа

```
from math import cos, exp, isfinite, sin
from pathlib import Path
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

def exact_y_plus_sin(x, x0, y0):
    p = -(sin(x) + cos(x)) / 2
    p0 = -(sin(x0) + cos(x0)) / 2
    return p + (y0 - p0) * exp(x - x0)

def exact_cos_minus_y(x, x0, y0):
    p = (cos(x) + sin(x)) / 2
    p0 = (cos(x0) + sin(x0)) / 2
    return p + (y0 - p0) * exp(x0 - x)

EQUATIONS = [
    ("y' = x + y", lambda x, y: x + y, lambda x, x0, y0: (y0 + x0 + 1) *
    exp(x - x0) - x - 1),
    ("y' = y - x^2 + 1", lambda x, y: y - x * x + 1, lambda x, x0, y0: (x +
    1) ** 2 + (y0 - (x0 + 1) ** 2) * exp(x - x0)),
    ("y' = x - y", lambda x, y: x - y, lambda x, x0, y0: x - 1 + (y0 - x0 +
    1) * exp(x0 - x)),
    ("y' = 2x - y", lambda x, y: 2 * x - y, lambda x, x0, y0: 2 * x - 2 + (y0
    - 2 * x0 + 2) * exp(x0 - x)),
    ("y' = y + sin(x)", lambda x, y: y + sin(x), exact_y_plus_sin),
    ("y' = cos(x) - y", lambda x, y: cos(x) - y, exact_cos_minus_y),
    ("y' = 2y", lambda x, y: 2 * y, lambda x, x0, y0: y0 * exp(2 * (x -
    x0))),
    ("y' = -3y", lambda x, y: -3 * y, lambda x, x0, y0: y0 * exp(-3 * (x -
    x0))),
]
```

```

    ("y' = xy", lambda x, y: x * y, lambda x, x0, y0: y0 * exp((x * x - x0 *
x0) / 2)),
    ("y' = y + x + 1", lambda x, y: y + x + 1, lambda x, x0, y0: -x - 2 + (y0
+ x0 + 2) * exp(x - x0)),
]

def improved_euler_step(f, x, y, h):
    predicted = y + h * f(x, y)
    return y + h * (f(x, y) + f(x + h, predicted)) / 2

def rk4_step(f, x, y, h):
    k1 = h * f(x, y)
    k2 = h * f(x + h / 2, y + k1 / 2)
    k3 = h * f(x + h / 2, y + k2 / 2)
    k4 = h * f(x + h, y + k3)
    return y + (k1 + 2 * k2 + 2 * k3 + k4) / 6

def get_steps_count(x0, xn, h):
    if xn <= x0:
        raise ValueError("xn должен быть больше x0")
    if h <= 0:
        raise ValueError("h должен быть положительным")
    steps = round((xn - x0) / h)
    if steps < 4:
        raise ValueError("интервал должен содержать минимум 4 шага")
    if steps > 10_000:
        raise ValueError("количество шагов не должно превышать 10_000")
    if abs(steps * h - (xn - x0)) > 1e-8 * max(1.0, abs(xn - x0)):
        raise ValueError("h должен целиком делить интервал [x0, xn]")
    return steps

def solve_by_step(f, x0, y0, xn, h, step_func):
    y = y0
    points = [(x0, y0)]
    for i in range(get_steps_count(x0, xn, h)):
        x = x0 + i * h
        y = step_func(f, x, y, h)
        points.append((x + h, y))
    return points

def solve_adams(f, x0, y0, xn, h, eps):
    y = y0
    points = [(x0, y0)]
    for i in range(3):
        x = x0 + i * h
        y = rk4_step(f, x, y, h)
        points.append((x + h, y))

    for i in range(3, get_steps_count(x0, xn, h)):
        fi = f(*points[i])
        fi1 = f(*points[i - 1])
        fi2 = f(*points[i - 2])
        fi3 = f(*points[i - 3])
        x_next = x0 + (i + 1) * h
        y = points[i][1] + h * (55 * fi - 59 * fi1 + 37 * fi2 - 9 * fi3) / 24

        for _ in range(50):
            y_next = points[i][1] + h * (9 * f(x_next, y) + 19 * fi - 5 * fi1
+ fi2) / 24
            if abs(y_next - y) <= eps:
                y = y_next
                break
        y = y_next

```

```

        points.append((x_next, y))
    return points

def runge_solve(name, order, step_func, f, x0, y0, xn, h, eps):
    result = None
    for _ in range(20):
        coarse = solve_by_step(f, x0, y0, xn, h, step_func)
        fine = solve_by_step(f, x0, y0, xn, h / 2, step_func)
        error = max(abs(coarse[i][1] - fine[2 * i][1]) / (2**order - 1) for i
in range(1, len(coarse)))
        result = {"name": name, "points": fine, "h": h / 2, "error": error,
"error_name": "Максимальная оценка по правилу Рунге"}
        if error <= eps:
            return result
        h /= 2
    return result

def exact_error(points, exact, x0, y0):
    return max(abs(exact(x, x0, y0) - y) for x, y in points)

def read_float(prompt):
    while True:
        try:
            value = float(input(prompt).replace(",", "."))
            if isfinite(value):
                return value
        except ValueError:
            pass
    print("Ошибка: введите число.")

def read_equation():
    print("Выберите ОДУ:")
    for i, (name, _, _) in enumerate(EQUATIONS, 1):
        print(f"{i}. {name}")

    while True:
        try:
            number = int(input("Номер уравнения: "))
            if 1 <= number <= len(EQUATIONS):
                return EQUATIONS[number - 1]
        except ValueError:
            pass
    print("Ошибка: введите номер из списка.")

def read_data():
    while True:
        x0 = read_float("x0: ")
        y0 = read_float("y0: ")
        xn = read_float("xn: ")
        h = read_float("h: ")
        eps = read_float("eps: ")
        try:
            get_steps_count(x0, xn, h)
            if eps <= 0:
                raise ValueError("eps должен быть положительным")
            return x0, y0, xn, h, eps
        except ValueError as error:
            print(f"Ошибка: {error}. Повторите ввод.")

def print_table(result, exact, x0, y0, eps):
    print(result["name"])

```

```

    print(f"{'i':>3} {'x':>12} {'у числ.':>16} {'у точн.':>16}
{'|ошибка|':>14}")
    for i, (x, y) in enumerate(result["points"]):
        true_y = exact(x, x0, y0)
        print(f"{'i':>3} {'x':>12.6f} {'y':>16.8f} {'true_y':>16.8f} {'abs(true_y -
y):>14.6g}")
    print(f"Использованный шаг: {result['h']:.8g}")
    print(f"{result['error_name']}: {result['error']:.8g}")
    print("Точность достигнута" if result["error"] <= eps else "Точность не
достигнута")
    print()

def save_plot(equation_name, exact, results, x0, y0):
    path = Path(__file__).with_name("solution_plot.png")
    fig, axes = plt.subplots(3, 1, figsize=(10, 12))
    fig.suptitle(equation_name)

    for axis, result in zip(axes, results):
        xs = [x for x, _ in result["points"]]
        ys = [y for _, y in result["points"]]
        exact_xs = [xs[0] + (xs[-1] - xs[0]) * i / 200 for i in range(201)]
        exact_ys = [exact(x, x0, y0) for x in exact_xs]

        axis.plot(exact_xs, exact_ys, label="Точное решение", color="black")
        axis.plot(xs, ys, marker="o", label=f"{result['name']},
h={result['h']:.5g}")
        axis.set_title(result["name"])
        axis.set_xlabel("x")
        axis.set_ylabel("y")
        axis.grid(True)
        axis.legend()

    fig.tight_layout()
    fig.savefig(path, dpi=160)
    plt.close(fig)
    print(f"График сохранен: {path}")

def main():
    equation_name, f, exact = read_equation()
    x0, y0, xn, h, eps = read_data()

    results = [
        runge_solve("Усовершенствованный метод Эйлера", 2,
improved_euler_step, f, x0, y0, xn, h, eps),
        runge_solve("Метод Рунге-Кутты 4-го порядка", 4, rk4_step, f, x0, y0,
xn, h, eps),
    ]

    adams_points = solve_adams(f, x0, y0, xn, h, eps)
    results.append({
        "name": "Метод Адамса",
        "points": adams_points,
        "h": h,
        "error": exact_error(adams_points, exact, x0, y0),
        "error_name": "Максимальная погрешность по точному решению",
    })

    print()
    for result in results:
        print_table(result, exact, x0, y0, eps)
    save_plot(equation_name, exact, results, x0, y0)

```

```
if __name__ == "__main__":  
    main()
```

Вывод:

Во время выполнения данной лабораторной работы я:

- 1) Научился решать задачу Коши для обыкновенных дифференциальных уравнений
- 2) Написал классную программу, Степан Дмитриевич оценил